Docker Basic Commands Handbook

Images, Containers, Volumes, Networks, Logs, Exec, Cleanup, Build, Save and Load

This PDF explains basic Docker commands in a beginner-friendly format. Each item includes the command, an example, and a clear description. Start with image and container commands, then practice volumes, networks, logs and cleanup.

Simple Mental Model

- **Image:** A package or template. Example: python:3.10, mysql:8.0, ubuntu:22.04, or your Streamlit app image.
- **Container:** A running instance created from an image. One image can create many containers.
- **Dockerfile:** A recipe used to build your own Docker image.
- **Volume:** Persistent storage. Useful for databases and files that should not be lost.
- **Network:** Communication layer between containers. Containers in the same custom network can talk using container names.

Docker Setup and Information Commands

1. Check Docker version

Command	docker --version
Example	docker --version
Description	Checks whether Docker is installed and shows the installed Docker version.

2. Check Docker system information

Command	docker info
Example	docker info
Description	Shows detailed Docker engine information such as containers, images, storage driver, OS and Docker root directory.

3. Check Docker disk usage

Command	docker system df
Example	docker system df
Description	Shows how much space Docker images, containers, volumes and build cache are using.

Docker Image Commands

4. Search image in Docker Hub

Command	docker search image_name
Example	docker search python
Description	Searches Docker Hub for public images. Use this before pulling images such as python, mysql or ubuntu.

5. Pull latest image

Command	docker pull image_name
Example	docker pull python
Description	Downloads the latest version of an image from Docker Hub to your local system.

6. Pull specific image version

Command	docker pull image_name:tag
Example	docker pull python:3.10
Description	Downloads a specific image version. The tag after colon identifies the version.

7. List local images

Command	docker images docker image ls
Example	docker images
Description	Displays all Docker images available locally with repository, tag, image ID, created date and size.

8. Remove image

Command	docker rmi image_name:tag
Example	docker rmi python:3.10
Description	Deletes a Docker image from the local system. The image must not be used by an existing container unless forced.

9. Force remove image

Command	docker rmi -f image_id
Example	docker rmi -f abc123
Description	Forcefully removes an image. Use carefully, especially if containers were created from the image.

10. Remove dangling images

Command	docker image prune
Example	docker image prune
Description	Removes dangling or unused intermediate images. Docker asks for confirmation.

11. Remove all unused images

Command	docker image prune -a
Example	docker image prune -a
Description	Removes all images not currently used by containers. Useful for cleanup, but use carefully.

12. Inspect image

Command	docker inspect image_name
Example	docker inspect python:3.10
Description	Shows detailed image metadata such as layers, environment variables, architecture, working directory and default command.

13. Check image history

Command	docker history image_name
Example	docker history python:3.10
Description	Shows the layers used to build an image. Useful for understanding Dockerfile steps and image size.

14. Tag or rename image

Command	docker tag old_image new_image
Example	docker tag diabetes-app:v1 diabetes-app:latest
Description	Adds another name or tag to an existing image. Docker does not truly rename; it creates an additional tag.

15. Build image from Dockerfile

Command	docker build -t image_name .
Example	docker build -t my-python-app .
Description	Builds a Docker image using the Dockerfile present in the current directory. The dot means current folder.

16. Build image with version tag

Command	docker build -t image_name:version .
Example	docker build -t diabetes-app:v1 .
Description	Builds an image with a specific tag or version such as v1, v2 or latest.

Docker Container Run Commands

17. Run container from image

Command	docker run image_name
Example	docker run hello-world
Description	Creates and starts a container from an image. If image is missing locally, Docker downloads it automatically.

18. Run Ubuntu container

Command	docker run ubuntu
Example	docker run ubuntu
Description	Runs an Ubuntu container. It may stop immediately if no long-running process is started.

19. Run interactive container

Command	docker run -it image_name
Example	docker run -it ubuntu
Description	Starts a container in interactive terminal mode. Use this when you want to enter the container shell.

20. Run Python shell container

Command	docker run -it python:3.10
Example	docker run -it python:3.10

Description	Starts Python directly inside the container and opens the Python prompt.

21. Run Python container with bash

Command	docker run -it python:3.10 bash
Example	docker run -it python:3.10 bash
Description	Opens a Linux bash shell inside the Python image. Useful to check Python, pip and files.

22. Run container with custom name

Command	docker run --name container_name image_name
Example	docker run -it --name ubuntu_test ubuntu
Description	Creates a container with your chosen name instead of a random Docker-generated name.

23. Run container in background

Command	docker run -d image_name
Example	docker run -d nginx
Description	Runs the container in detached or background mode. Common for services like Nginx, MySQL, Streamlit and APIs.

24. Run with port mapping

Command	docker run -p host_port:container_port image_name
Example	docker run -p 8080:80 nginx
Description	Maps a port from your local machine to the container. Example: localhost:8080 reaches port 80 inside the container.

25. Run with name and port

Command	docker run -d --name name -p host_port:container_port image_name
Example	docker run -d --name mynginx -p 8080:80 nginx
Description	Runs a named background container with port mapping.

26. Run and auto-remove after exit

Command	docker run --rm image_name
Example	docker run --rm hello-world
Description	Automatically removes the container after it finishes. Good for temporary testing.

27. Run command with working directory and mount

Command	docker run -v local_path:/app -w /app image command
Example	docker run -v C:\Users\Shabarinath\myapp:/app -w /app python:3.10 python app.py
Description	Mounts a local folder, sets working directory inside container and runs a command such as python app.py.

Container Listing and Lifecycle Commands

28. List running containers

Command	docker ps
Example	docker ps
Description	Shows only currently running containers, including container ID, image, status, ports and names.

29. List all containers

Command	docker ps -a
Example	docker ps -a
Description	Shows all containers, including running, stopped and exited containers.

30. Start stopped container

Command	docker start container_name
Example	docker start mynginx
Description	Starts a container that already exists but is currently stopped.

31. Stop running container

Command	docker stop container_name
Example	docker stop mynginx
Description	Stops a running container safely.

32. Restart container

Command	docker restart container_name
Example	docker restart mynginx
Description	Stops and starts a container again. Useful when an application gets stuck.

33. Pause container

Command	docker pause container_name
Example	docker pause mynginx
Description	Freezes processes inside the container without stopping it.

34. Unpause container

Command	docker unpause container_name
Example	docker unpause mynginx
Description	Resumes a paused container.

35. Rename container

Command	docker rename old_name new_name
Example	docker rename mynginx nginx_server
Description	Changes the name of an existing container.

36. Remove stopped container

Command	docker rm container_name
Example	docker rm mynginx
Description	Deletes a stopped container. Stop it first if it is running.

37. Force remove running container

Command	docker rm -f container_name
Example	docker rm -f mynginx
Description	Forcefully stops and removes a container. Use carefully.

38. Remove all stopped containers

Command	docker container prune
Example	docker container prune
Description	Deletes all stopped containers after confirmation.

39. Remove all containers

Command	docker rm -f $(docker ps -aq)
Example	docker rm -f $(docker ps -aq)
Description	Removes all containers, both running and stopped. Very useful for reset, but be careful.

Working Inside Containers

40. Enter running container using bash

Command	docker exec -it container_name bash
Example	docker exec -it myubuntu bash
Description	Opens bash inside a running container. The container must already be running.

41. Enter running container using sh

Command	docker exec -it container_name sh
Example	docker exec -it mynginx sh
Description	Use sh when bash is not available inside minimal images such as Alpine or Nginx.

42. Run command inside container

Command	docker exec container_name command
Example	docker exec mynginx ls
Description	Runs a command inside the container without fully entering the shell.

43. View container logs

Command	docker logs container_name
Example	docker logs diabetes_app
Description	Shows application output and error logs from a container. Very important for debugging.

44. Follow live logs

Command	docker logs -f container_name
Example	docker logs -f diabetes_app
Description	Streams logs continuously. Press CTRL+C to stop watching.

45. Inspect container

Command	docker inspect container_name
Example	docker inspect mynginx
Description	Shows detailed container configuration: IP, ports, mounts, network, environment variables and status.

46. Check container resource usage

Command	docker stats
Example	docker stats
Description	Displays live CPU, memory, network and disk I/O usage for running containers.

47. Check processes inside container

Command	docker top container_name
Example	docker top mynginx
Description	Shows the processes running inside a container.

48. Check container IP address

Command	docker inspect -f "{{range .NetworkSettings.Networks}} {{.IPAddress}}{{end}}" container_name
Example	docker inspect -f "{{range .NetworkSettings.Networks}} {{.IPAddress}}{{end}}" mynginx
Description	Prints the internal Docker IP address of a container.

Copy Files Between Local System and Container

49. Copy local file to container

Command	docker cp local_file container_name:/path_inside_container
Example	docker cp app.py myubuntu:/app/app.py
Description	Copies a file from your local system into a container.

50. Copy file from container to local system

Command	docker cp container_name:/path_inside_container local_path
Example	docker cp myubuntu:/app/output.txt .
Description	Copies a file from a container to your local machine.

Save, Export, Import and Load

51. Commit container as new image

Command	docker commit container_name new_image_name
Example	docker commit myubuntu ubuntu-with-python
Description	Saves the current state of a container as a new Docker image. Useful after installing packages or adding files manually.

52. Run from committed image

Command	docker run -it new_image_name
Example	docker run -it ubuntu-with-python
Description	Creates a new container from your committed image.

53. Export container as tar

Command	docker export container_name -o file_name.tar
Example	docker export myubuntu -o myubuntu.tar
Description	Exports the container filesystem as a tar file. This is filesystem export, not full image history.

54. Import tar as image

Command	docker import file_name.tar new_image_name
Example	docker import myubuntu.tar ubuntu_imported:v1
Description	Imports an exported container filesystem as a new Docker image.

55. Save image as tar

Command	docker save image_name -o file_name.tar
Example	docker save diabetes-streamlit-app -o diabetes-app.tar
Description	Saves a Docker image to a tar file while preserving image layers and metadata.

56. Load image from tar

Command	docker load -i file_name.tar
Example	docker load -i diabetes-app.tar
Description	Loads a saved Docker image tar file back into Docker.

Volume Commands

57. Create volume

Command	docker volume create volume_name
Example	docker volume create mysql_data
Description	Creates a persistent Docker volume. Important for database storage.

58. List volumes

Command	docker volume ls

Example	docker volume ls
Description	Shows all Docker volumes.

59. Inspect volume

Command	docker volume inspect volume_name
Example	docker volume inspect mysql_data
Description	Shows volume details and internal mount location.

60. Use volume with container

Command	docker run -v volume_name:/container/path image_name
Example	docker run -d --name mysql_container -e MYSQL_ROOT_PASSWORD=root123 -v mysql_data:/var/lib/mysql mysql:8.0
Description	Mounts a Docker volume into a container. Data remains even if the container is removed.

61. Remove volume

Command	docker volume rm volume_name
Example	docker volume rm mysql_data
Description	Deletes a Docker volume. Be careful because database data may be lost.

Bind Mount Commands

62. Mount local folder to container

Command	docker run -v local_path:container_path image_name
Example	docker run -it -v C:\Users\Shabarinath\myapp:/app python:3.10 bash
Description	Connects a local folder to a folder inside the container. Useful for development because local file changes appear inside the container.

Environment Variable Commands

63. Pass environment variable

Command	docker run -e VARIABLE_NAME=value image_name
Example	docker run -e MYSQL_ROOT_PASSWORD=root123 mysql:8.0
Description	Passes configuration values such as passwords, API keys, database names or model paths into the container.

64. Pass multiple environment variables

Command	docker run -e VAR1=value1 -e VAR2=value2 image_name
Example	docker run -d --name mysql_container -e MYSQL_ROOT_PASSWORD=root123 -e MYSQL_DATABASE=mydb -e MYSQL_USER=admin -e MYSQL_PASSWORD=admin123 mysql:8.0

Description	Passes multiple environment variables to the same container.

Network Commands

65. List Docker networks

Command	`docker network ls`
Example	`docker network ls`
Description	Shows Docker networks such as bridge, host and none, plus your custom networks.

66. Create custom network

Command	`docker network create network_name`
Example	`docker network create app_network`
Description	Creates a custom network so containers can communicate using container names.

67. Run container in network

Command	`docker run --network network_name image_name`
Example	`docker run -d --name mysql_db --network app_network -e MYSQL_ROOT_PASSWORD=root123 mysql:8.0`
Description	Starts a container connected to a specific Docker network.

68. Connect existing container to network

Command	`docker network connect network_name container_name`
Example	`docker network connect app_network mynginx`
Description	Connects an already existing container to a Docker network.

69. Remove network

Command	`docker network rm network_name`
Example	`docker network rm app_network`
Description	Deletes a Docker network. Containers must be disconnected first.

Docker System Cleanup Commands

70. Basic system prune

Command	`docker system prune`
Example	`docker system prune`
Description	Removes stopped containers, unused networks, dangling images and build cache.

71. Full prune including volumes

Command	`docker system prune -a --volumes`
Example	`docker system prune -a --volumes`
Description	Removes unused images, stopped containers, unused

	networks, unused volumes and build cache. Use carefully.

Most Important Daily-use Commands

```
docker images                  # show images
docker ps                      # show running containers
docker ps -a                   # show all containers
docker pull python:3.10        # download image
docker run -it ubuntu          # run Ubuntu interactively
docker build -t diabetes-app .     # build image from Dockerfile
docker run -d --name app -p 8501:8501 diabetes-app   # run app container
docker logs app                # see logs
docker exec -it app bash       # enter container
docker stop app                # stop container
docker start app               # start container
docker rm app                  # remove container
docker rmi diabetes-app        # remove image
docker commit app app-backup:v1    # save container as image
```

Recommended Practice Flow for Your Streamlit / ML App

1. Keep app.py, requirements.txt, diabetes.pkl and Dockerfile in one folder.
2. Build the image:
   docker build -t diabetes-app .
3. Run the container:
   docker run -d --name diabetes_container -p 8501:8501 diabetes-app
4. Check running container:
   docker ps
5. Check app logs:
   docker logs diabetes_container
6. Enter the container if needed:
   docker exec -it diabetes_container bash
7. Stop and start again:
   docker stop diabetes_container
   docker start diabetes_container